

Sentinel-Z: Predictive Behavioural-Drift Containment for Autonomous Cloud Agents via a Hybrid Learned-Observation POMDP

Sufiyan Khan S

Dept. of Information Science &
Engg.

The Oxford College of
Engineering
Bengaluru, India

sufiyanbitwise799@gmail.com

Prathiba G

Dept. of Information Science &
Engg.

The Oxford College of
Engineering
Bengaluru, India

gprathiba776@gmail.com

Shreeya Singh

Dept. of Information Science &
Engg.

The Oxford College of
Engineering
Bengaluru, India

shreeyasinghrajput7@gmail.com

Vachanashree G S

Dept. of Information Science &
Engg.

The Oxford College of
Engineering
Bengaluru, India

vachanasiddesh05@gmail.com

Mr. Yadhu Krishna M R

Project Guide, Assistant Professor, Dept. of Information Science & Engineering

The Oxford College of Engineering, Bengaluru, India — mr_yadhukrishna.is@theoxford.edu

Abstract—Autonomous agents built on large language models increasingly hold live cloud credentials and act on content they retrieve at run time, creating an indirect prompt-injection surface that perimeter controls and static least-privilege policies do not address. Existing runtime defences decide on each tool call in isolation and intervene inside the agent, so a blocked call leaves the compromised agent free to pursue an alternative route with credentials still valid. This paper presents Sentinel-Z, an inline zero-trust gateway that treats behavioural drift as a trajectory rather than a sequence of independent events. Five lightweight signals—*injection likelihood, task alignment, privilege delta, taint propagation and sequence novelty*—are condensed by a learned logistic hazard model into a discrete observation symbol that drives a Bayes belief filter over five behavioural states, two of which are absorbing. A finite-horizon policy obtained by grid value iteration selects one of five graduated actions, and containment is applied at the credential layer by revoking the session capability token rather than refusing a single call. Conditioning the transition matrix on the passive action yields an absorbing Markov chain from which the N -step probability of reaching the harmful state is obtained in closed form, giving advance warning before the harmful call is issued. The system is evaluated on the AgentDojo benchmark under both static and adaptive attackers, with every decision recorded in a hash-chained, tamper-evident log.

Keywords—zero trust architecture, autonomous AI agents, indirect prompt injection, POMDP, behavioural drift, capability revocation, cloud security, adaptive adversary

I. Introduction

Large language models have made it practical to deploy autonomous agents that plan and execute multi-step tasks against real infrastructure. Such agents are routinely granted credentials to read object storage, retrieve configuration parameters and fetch secrets, because those permissions are necessary for the work they are asked to do. The security consequence is that the agent is simultaneously a privileged principal and a component whose behaviour is determined at run time by content it did not author.

Indirect prompt injection exploits precisely this property. An instruction embedded in a document, e-mail or calendar entry that the agent retrieves in the ordinary course of its task

can redirect the agent toward actions the user never requested. No credential is stolen and no perimeter is crossed; the legitimate principal is simply persuaded to act. Static least-privilege policies do not help, because the agent needs the permissions it is being induced to misuse.

A body of runtime defences has emerged in response, including policy-enforcement proxies, dataflow-label systems and verified-code approaches. Two limitations recur. First, decisions are made on each tool call in isolation, so a defence answers “is this call permitted” rather than “where is this session heading”. Second, intervention happens inside the agent—rejecting a call, withholding a tool, injecting a warning—leaving the session credential valid, so a blocked agent may simply attempt a different route to the same objective.

This paper takes a different position on both points. Sentinel-Z models the session as a partially observable process and estimates, at every step, the probability that the trajectory terminates in harm within a bounded horizon. When intervention is warranted it acts at the credential layer, revoking the capability token so that the entire class of subsequent action fails rather than the single call that triggered the decision.

The contributions are as follows. (i) A session-level behavioural drift model in which a learned discriminative model serves as the observation encoder for a belief-state controller, so that prediction and control share one estimated transition matrix. (ii) Enforcement by capability revocation rather than call refusal, evaluated by blast radius rather than block rate alone. (iii) Evaluation under an adaptive adversary that rewrites its injection after each failure, reporting attack success as a function of adaptation rounds rather than as a single static figure. (iv) A graduated response set with the utility cost of each action measured explicitly.

II. Related Work

A. Zero Trust for Agentic Systems

Zero trust architectures replace perimeter assumptions with continuous verification of every access request. Recent work extends the model to agentic settings using decentralised identifiers, verifiable credentials and trust-adaptive runtime environments, and identifies logic-layer prompt injection as a distinct vulnerability class [3].

Behaviour-based dynamic access control has likewise been proposed, building user profiles and trust levels from observed activity [6]. These architectures establish the enforcement vocabulary this work adopts, but they evaluate access decisions rather than trajectories.

B. Probabilistic Runtime Monitoring

Several systems learn probabilistic models of agent execution to anticipate unsafe behaviour. ProbGuard learns a discrete-time Markov chain from execution traces and predicts safety violations ahead of time, reporting substantial reductions in unsafe behaviour while preserving most task completion [7]. SafetyDrift models safety trajectories as absorbing Markov chains with closed-form absorption analysis, reporting detection several steps in advance at a small fraction of the cost of using a secondary language model as a judge [5]. Convergence properties of verifier pipelines have also been established using sequential absorbing chains [13]. Sentinel-Z shares the absorbing-chain formulation for its prediction claim, but couples it to a belief filter over partially observed states and to an explicit intervention policy.

C. Policy Enforcement and Formal Guarantees

Policy-based systems enforce least privilege at the level of individual tool calls, typically integrating as a proxy so that agent code is unmodified [15], [16]. Verified-code approaches provide formal guarantees by checking generated policies offline and monitoring actions against them at run time [8]. Evaluations of adherence to hierarchical safety principles have shown a measurable cost of compliance, in which safety constraints degrade task performance—a trade-off this work reports directly [9].

D. Benchmarks and Adaptive Red Teaming

Standardised environments now exist for evaluating agent security jointly with utility, providing realistic user tasks together with injection cases across several domains [14], [17]. Contemporary red-teaming practice has moved toward autonomous, agent-orchestrated attack generation, in which an attacker model selects strategies and escalates across turns [1]. Evaluation against fixed attack suites therefore risks overstating robustness, motivating the adaptive protocol used here.

III. Threat Model

The adversary controls content that the agent will retrieve during a legitimate task but does not control the agent process, the gateway or the credential broker. The adversary knows the defence architecture, and in the adaptive setting receives feedback indicating which signal contributed most to a block, permitting targeted refinement of subsequent attempts. The adversary cannot read the capability token directly and cannot invoke the broker administrative interface.

The trusted computing base comprises the gateway, the capability broker and the evidence log. The agent process and

its context window are explicitly untrusted. The objective of the defence is to prevent completion of a harmful action while preserving the agent's ability to complete benign tasks; a defence that blocks all activity is a failure by this criterion.

IV. System Architecture

The architecture separates concerns into four planes, shown in Fig. 1. The agent plane hosts the monitored agent and the environment it acts upon. The interception plane captures every tool invocation inline, subject to a hard decision budget after which the call is denied by default, so that failure of the analytics path cannot silently permit an action.

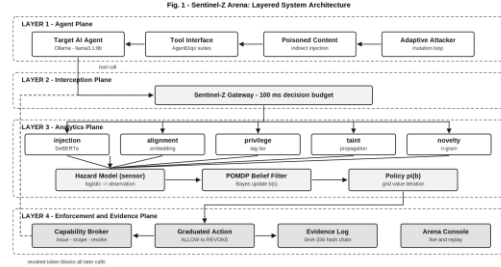


Fig. 1. Layered system architecture.

The analytics plane computes the five signals and executes the belief update and policy lookup described in Section V. The enforcement and evidence plane applies the selected action through the capability broker and appends a record to the hash-chained log. Because the broker owns the session token, revocation invalidates every subsequent call rather than only the call under evaluation.

Fig. 2 shows the state machine over which the belief is maintained. The harmful and contained states are absorbing; the state reported to an operator is the arg max of the belief distribution and is presented as such, not as a certainty.

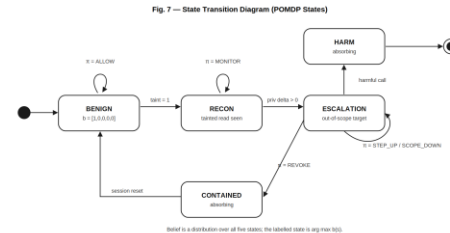


Fig. 2. Behavioural state model. Harmful and contained states are absorbing.

V. Method

A. Risk Signals

Five signals are computed per call. Injection likelihood scores the most recent tool result with a transformer sequence classifier, targeting instruction-like content in retrieved data. Task alignment measures the cosine distance between an embedding of the pending call and an embedding of the user task, penalising tools outside the task's expected set. Privilege delta is the ordinal difference between the sensitivity tier of

the target resource and the maximum tier the task requires, with tiers declared as resource tags rather than hard-coded identifiers. Taint is a propagating set: entities extracted from any tool result exceeding the injection threshold are marked, and any later call whose arguments intersect that set is tainted. Sequence novelty is the Laplace-smoothed novelty of the tool-name trigram against benign execution history.

Taint is the signal that distinguishes reading poisoned content from acting upon it, and is therefore the principal contributor to advance warning.

B. Observation Encoding

A logistic model is fitted over the five signals together with their running maxima and slopes, predicting whether a harmful action occurs within the next k steps. Sessions are split by session and in time order; splitting by step would leak the outcome into the features. The scalar output is quantised into five bins whose edges are fitted once on a benign validation split, and combined with the binary taint indicator to give an observation alphabet of ten symbols. Keeping the alphabet small is what renders the subsequent belief computation tractable and its estimated matrices interpretable.

C. Belief Update

Let S denote the five states and O the ten observation symbols. The transition tensor $T[s][a][s']$ and observation matrix $O[s'][o]$ are estimated from labelled traces with Laplace smoothing; rows corresponding to absorbing states are fixed. Given the previous action a and the current observation o , the belief is updated in two steps, applied once per call and in this order:

$$\tilde{b}(s') = \sum_s T[s][a][s'] \cdot b(s) \quad (1)$$

$$b'(s') = \eta \cdot O[s'][o] \cdot \tilde{b}(s') \quad (2)$$

where η normalises the distribution. Where the normaliser falls below a numerical floor the predicted belief is retained and the event counted, avoiding division by a vanishing quantity.

D. Policy

Action costs are assigned to the five responses and a single cost is applied on the transition into the harmful state, expressed as a ratio of breach cost to the cost of wrongly revoking a benign session. Costs are state-independent, so the benefit of intervening derives from the estimated transition function rather than from a hand-tuned reward, which reduces the number of free parameters and the scope for fitting the policy to the desired conclusion. The policy is obtained offline by backward induction over a regular grid on the five-dimensional belief simplex and applied at run time as a nearest-neighbour lookup, so no optimisation occurs in the decision path.

E. Prediction

Conditioning T on the passive monitoring action yields an absorbing Markov chain. Reordering into canonical form and

computing the fundamental matrix gives the probability of absorption into the harmful state within N steps, and the expected number of steps to absorption, in closed form. The prediction and control claims therefore rest on a single estimated transition matrix rather than on two independently fitted models.

F. Enforcement

Five graduated actions are available: allow, monitor, scope-down, step-up and revoke. Scope-down forwards a restricted form of the call, redacting fields or capping returned volume, and is the action under which utility and security are simultaneously preserved. Revocation invalidates the session capability token at the broker, so that all subsequent calls fail irrespective of type.

G. Evidence

Each decision record contains the signals, the observation, the belief vector, the selected action and the SHA-256 hash of the preceding record, serialised canonically. The chain is verifiable offline, and modification of any record is localised to that record by verification.

VI. Implementation

The system is implemented in Python. The target agent is driven by an open-weights model served locally, and the environment and injection corpus are drawn from an established agent-security benchmark providing realistic user tasks together with injection cases across several domains [14]. The injection classifier and sentence embedder are small transformer models executed on CPU; no component of the decision path issues a network request or invokes a language model, which is what permits the decision budget to be met.

The capability broker is implemented behind an interface with two backends: a simulated in-process broker used for reproducible evaluation, and a cloud backend that issues short-lived credentials and applies revocation by attaching an explicit deny policy conditioned on token issue time. Results reported here use the simulated backend so that the artifact is reproducible without cloud credentials.

The evaluation harness runs matched pairs of defended and undefended sessions on identical seeds and records every decision, permitting exact replay without a model or a network.

VII. Evaluation

A. Experimental Setup

Two benchmark suites are used. Sessions are collected for benign tasks and for injection tasks under each attack template. Splits are by session and in time order. All figures are reported as mean and standard deviation over five random seeds. The model tier is stated explicitly: the target agent uses an open-weights model of approximately eight billion parameters, representing cost-constrained self-hosted deployments in which base-model robustness cannot be assumed.

B. Baselines

Four comparators are implemented against identical traces: no defence; a threshold on the injection classifier alone; a per-call policy enforcing a static allowlist derived from the task, without session state; and the full system. A secondary-model judge is additionally measured for latency.

C. Results

VIII. Limitations

Several limitations constrain the generality of these results. Transition and observation matrices are estimated from a bounded number of labelled sessions, and several cells are sparse; raw counts are reported alongside smoothed probabilities so that this is visible. Evaluation uses simulated environments rather than production infrastructure, and the capability broker is simulated in the reported configuration. The target model is a single open-weights model of one size class, and stronger base models are known to resist single-context injections without any defence, so the improvement reported here should not be extrapolated to frontier models. The adaptive attacker is itself a language model and its search is neither exhaustive nor guaranteed to find the strongest available attack. Finally, the model assumes stationary transition dynamics within a session.

IX. Conclusion

Sentinel-Z treats agent compromise as a trajectory to be anticipated rather than a call to be filtered, and contains it at the credential layer rather than at the point of the individual request. Coupling a learned observation encoder to a belief-state controller allows prediction and control to share one estimated transition matrix, and yields advance warning in closed form. Evaluation under an adaptive adversary, rather than a fixed suite, is argued to be the appropriate robustness criterion for this class of defence. Future work includes multi-cloud telemetry, agent-to-agent trust propagation and evaluation across model scales.

References

- [1] M. Mohammadi, Y. Li, J. Lo, and W. Yip, “Evaluation and benchmarking of LLM agents: A survey,” in Proc. ACM SIGKDD, Toronto, Canada, 2025.
- [2] A. Mahida, “An intellectual zero trust security framework using deep reinforcement learning for predictive threat mitigation,” IEEE Access, vol. 14, pp. 24602–24618, 2026.
- [3] K. Huang et al., “Fortifying the agentic web: A unified zero-trust architecture against logic-layer threats,” arXiv:2508.12259, 2025.
- [4] H. Sharma, “Behavioral analytics and zero trust,” Int. J. Inf. Technol. Manag. Inf. Syst., vol. 12, no. 1, pp. 63–84, 2021.
- [5] A. Dhodapkar and F. Pishori, “SafetyDrift: Predicting when AI agents cross the line before they actually do,” arXiv:2603.27148, 2026.
- [6] Q. Yao, Q. Wang, X. Zhang, and J. Fei, “Dynamic access control and authorization system based on zero-trust architecture,” in Proc. ACM CCRIS, Xiamen, China, 2020.
- [7] H. Wang, C. M. Poskitt, J. Wei, and J. Sun, “ProbGuard: Probabilistic runtime monitoring for LLM agent safety,” under submission, 2026.
- [8] L. Miculicich et al., “VeriGuard: Enhancing LLM agent safety via verified code generation,” arXiv:2510.05156, 2025.
- [9] R. Potham, “Evaluating LLM agent adherence to hierarchical safety principles,” in Proc. TAIG Workshop, ICML, Vancouver, Canada, 2025.
- [10] R. Kapoor and M. Kas, “Breaking the observability tax: Dynamic resolution anomaly detection via topology-aware active LLM agents,” IEEE Access, vol. 14, 2026.
- [11] J. Yang et al., “SwarmSense-DNN: A trustworthy and decentralized neural framework for proactive anomaly defense in consumer IoT,” IEEE Trans. Consum. Electron., vol. 72, no. 1, 2026.
- [12] P. Talli et al., “Pragmatic communication for remote control of finite-state Markov processes,” IEEE J. Sel. Areas Commun., vol. 43, no. 7, 2025.
- [13] P. Dantas, L. Cordeiro, Y. Sun, and W. Junior, “The 4/8 bound: Designing predictable LLM-verifier systems for formal method guarantee,” arXiv:2512.02080, 2025.
- [14] E. Debenedetti et al., “AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents,” in Proc. NeurIPS Datasets and Benchmarks, 2024.
- [15] T. Shi et al., “Progent: Programmable privilege control for LLM agents,” arXiv:2504.11703, 2025.
- [16] E. Debenedetti et al., “Defeating prompt injections by design,” arXiv:2503.18813, 2025.
- [17] X. Zhan et al., “InjecAgent: Benchmarking indirect prompt injections in tool-integrated LLM agents,” in Findings of ACL, 2024.